

BASE TRADING BOT BLUEPRINT

Rust-Centric Low-Latency Architecture for Base Network

Versi 2.0 | September 2026

Revisi ini mengganti core execution dari arsitektur Node.js-centric menjadi Rust-centric. Rust digunakan pada jalur kritis yang sensitif terhadap latency, sedangkan Next.js/TypeScript dipertahankan untuk dashboard dan control plane.

Tujuan: membangun bot trading Base yang event-driven, cepat, modular, dapat diaudit, dan memiliki risk control sebelum setiap eksekusi.

Catatan: blueprint ini adalah rancangan teknis. Trading aset kripto dapat menimbulkan kerugian besar; tidak ada strategi yang menjamin keuntungan.

Daftar Isi

1. Ringkasan Revisi v2.0
2. Arsitektur Target Rust-Centric
3. Critical Path dan Low-Latency Design
4. Data Ingestion, Flashblocks, dan Market State
5. Strategy Engine
6. Fast-Profit Strategy V1
7. Risk Engine dan Trade Validation
8. Simulation dan Execution Engine
9. Wallet / Signer Security
10. Stack Teknologi
11. Service dan Crate Layout Rust
12. Database, Messaging, API, dan Dashboard
13. Observability, Fail-Safe, dan Performance Budget
14. Roadmap MVP ke Production
15. Checklist Implementasi

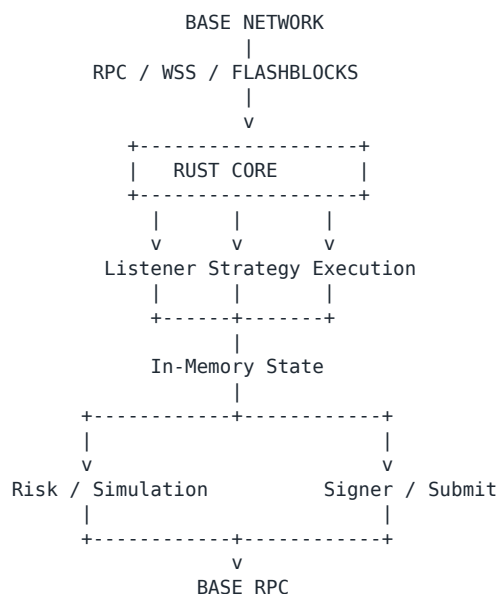
1. Ringkasan Revisi v2.0

Versi 2.0 mempertahankan prinsip blueprint awal - intelligence, risk, strategy, simulation, execution, portfolio - tetapi memindahkan komponen yang berada di jalur eksekusi ke Rust agar latency lebih predictable dan concurrency lebih efisien.

Komponen	v1.0	v2.0
Dashboard	Next.js	Next.js / TypeScript
Blockchain listener	Node.js	Rust + Tokio + Alloy
Market state	Redis-heavy	Rust in-memory + Redis durability
Strategy engine	Node.js/Python	Rust
Risk engine	Node.js	Rust
Simulation	Node.js	Rust
Execution + signer	Node.js	Rust
Messaging	Redis/BullMQ	NATS / Redis Streams / Kafka
Analytics	Node/Python	Rust + optional Python research

Prinsip utama: bahasa bukan satu-satunya sumber kecepatan. RPC latency, state locality, parallelism, quote/simulation design, dan signing path sering lebih menentukan daripada perbedaan runtime beberapa milidetik.

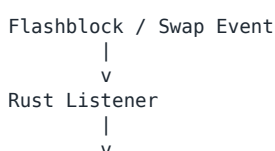
2. Arsitektur Target Rust-Centric



Control Plane: Next.js / TypeScript Dashboard
Persistence : PostgreSQL + Redis
Messaging : NATS / Redis Streams / Kafka

Control plane tidak berada di critical path. Dashboard dapat restart atau mengalami spike tanpa menghentikan event listener dan trading engine selama core service tetap sehat.

3. Critical Path dan Low-Latency Design



```
Update MarketState (memory)
  |
  v
Strategy Score
  |
  v
Risk Gate
  |
  v
Parallel Simulation / Quote
  |
  v
Local Signer
  |
  v
Submit Transaction
```

Aturan desain

- Tidak melakukan RPC read yang sama berulang kali jika state sudah tersedia lokal.
- Gunakan bounded channels dan backpressure agar burst event tidak merusak stabilitas.
- Pisahkan hot path dari persistence; database write tidak boleh memblokir keputusan trading.
- Gunakan parallel RPC untuk quote/simulation yang independen.
- Simpan nonce, allowance, balance, pool state, dan token metadata dalam cache terkontrol.

4. Data Ingestion, Flashblocks, dan Market State

Listener membaca block/log/transaction/DEX event dan, bila provider mendukung, stream pre-confirmed/Flashblocks. Semua event dinormalisasi menjadi event internal sebelum masuk ke market engine.

```
BASE -> Rust Listener -> Decoder -> Event Bus -> MarketState
                                     |
                                     +-> Token Intelligence
                                     +-> Whale Intelligence
                                     +-> Strategy Engine
```

In-memory MarketState

```
MarketState {
    token, pool, block_number,
    price, liquidity, reserves,
    volume_1m, volume_5m, volume_1h,
    buys_1m, sells_1m,
    price_change_1m, price_change_5m,
    whale_netflow,
    gas_state,
    last_update
}
```

Redis digunakan sebagai cache/durability antar service, tetapi keputusan trading tidak sebaiknya memerlukan roundtrip Redis untuk setiap event bila semua state kritis sudah berada di proses Rust.

5. Strategy Engine

Strategi	Sinyal utama	Horizon	Catatan
Momentum	price + volume + buy pressure	detik-menit	utama untuk fast-profit
Volume Breakout	volume spike vs baseline	detik-menit	perlu konfirmasi harga/likuiditas
Liquidity Growth	TVL/reserve meningkat	menit-jam	hindari liquidity palsu/sementara
Whale / Smart Money	net buy wallet terpilih	detik-menit	bukan copy trade buta
New Pool	pool/token baru	detik-menit	risk sangat tinggi
Mean Reversion	deviasi dari mean	menit	lebih cocok pair likuid
DEX Arbitrage	price dislocation	sub-detik-detik	sangat sensitif biaya/latency
Custom Multi-factor	gabungan faktor	fleksibel	direkomendasikan

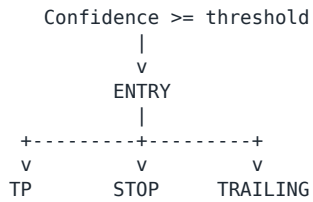
Multi-factor score

```
Momentum      22/25
Volume         18/20
Liquidity      17/20
Whale          13/15
Token Safety   18/20
-----
Total          88/100 -> candidate
```

6. Fast-Profit Strategy V1

Strategi fast-profit bukan jaminan profit cepat. Tujuannya adalah menangkap pergerakan pendek dengan loss cap yang kecil dan execution discipline.

```
Momentum + Volume Breakout + Liquidity Growth
      +
      Smart Money Flow
      +
      Risk Filter
      +
      Buy/Sell Simulation
      |
      v
```

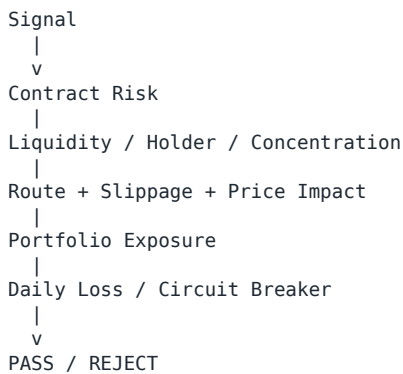


Contoh parameter awal untuk backtest, bukan default live

Parameter	Contoh
Liquidity minimum	\$250k-\$300k
Volume spike	$\geq 2 \times$ baseline
Buy/Sell ratio	> 1.5
Risk score	≥ 80
Price impact	$< 0.3\%$
TP	dinamis, mis. $+0.8\%$ / $+1.5\%$
Stop	mis. -0.5%
Max daily loss	hard lock, mis. 3%

Parameter wajib diuji dengan paper trading dan data historis. Jangan langsung menganggap angka di atas optimal.

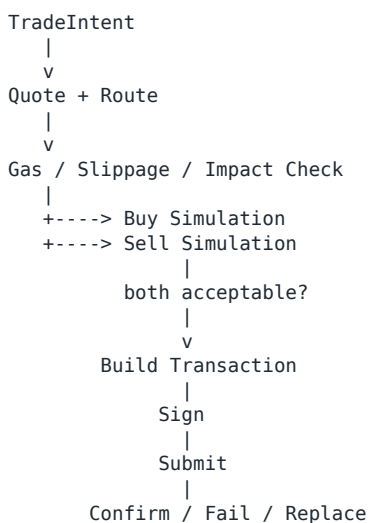
7. Risk Engine dan Trade Validation



Kategori	Contoh pemeriksaan
Contract	sellability, transfer behavior, suspicious permissions
Liquidity	depth, reserve, sudden removal, impact
Holders	top-holder concentration, deployer allocation
Execution	gas, route, quote age, slippage
Portfolio	position size, total exposure, correlation
System	RPC health, stale state, sequence gap

8. Simulation dan Execution Engine

Tidak ada auto-buy langsung dari strategy signal. Semua transaksi melalui validation dan simulation.



Low-latency optimizations

- Precompute router calldata bila pola transaksi berulang.
- Cache token decimals, pool addresses, router metadata, approvals, dan nonce dengan invalidation yang benar.
- Quote/simulation independen dijalankan paralel bila aman.
- Gunakan time-to-live untuk quote agar bot menolak data yang stale.
- Pisahkan executor dari UI dan reporting.

9. Wallet dan Signer Security

Trading Core -> unsigned tx -> Isolated Signer -> signed tx -> RPC

Database stores: wallet address, trades, tx hash

Database does NOT store: plaintext private key

- Gunakan Vault/KMS/HSM atau encrypted key store.
- Signer service memiliki permission minimal dan network policy ketat.
- Batasi maximum transaction value dan approved contract/router.
- Tambahkan emergency revoke / trading lock.
- Jangan log private key, seed phrase, atau raw secret.

10. Stack Teknologi

Layer	Pilihan utama	Alasan
Frontend	Next.js + TypeScript	produktif untuk dashboard/control plane
Core runtime	Rust	latency predictable, memory safety
Async	Tokio	concurrency/network I/O
EVM library	Alloy	Ethereum/EVM interaction di Rust
HTTP/WSS	reqwest / Alloy transports	RPC dan stream
Messaging	NATS	low-latency service messaging
Cache	Rust memory + Redis	hot state + shared cache
Database	PostgreSQL	persistence dan analytics
Research	Python optional	backtesting/statistical research
Metrics	Prometheus + Grafana	observability
Logs	Loki / structured tracing	debugging
Deploy	Docker	isolasi dan reproducibility

Rekomendasi: jangan menjadikan microservices sebagai tujuan. Untuk MVP, satu Rust process modular dapat lebih cepat dan sederhana; pecah menjadi service hanya ketika scaling/fault isolation membutuhkannya.

11. Service dan Crate Layout Rust

```
base-bot/  
|-- Cargo.toml  
|-- crates/  
|   |-- chain/  
|   |   |-- rpc  
|   |   |-- websocket  
|   |   |-- flashblocks  
|   |-- decoder/  
|   |-- market/  
|   |-- token_intel/  
|   |-- dex/  
|   |   |-- uniswap  
|   |   |-- aerodrome  
|   |   |-- aggregator  
|   |-- strategy/  
|   |   |-- momentum  
|   |   |-- breakout  
|   |   |-- whale  
|   |   |-- liquidity  
|   |   |-- arbitrage  
|   |-- risk/  
|   |-- simulator/  
|   |-- execution/  
|   |-- signer/  
|   |-- portfolio/  
|-- apps/  
|   |-- scanner  
|   |-- trader  
|   |-- api
```

Gunakan domain types yang jelas seperti TradeIntent, Signal, RiskDecision, Quote, SimulationResult, dan ExecutionReport agar modul dapat diuji secara independen.

12. Database, Messaging, API, dan Dashboard

Database inti

tokens, pools, blocks, swaps, prices, liquidity_snapshots
wallets, positions, orders, trades
strategies, signals, risk_scores, risk_events
alerts, system_logs

Event bus

block.event
swap.event
pool.event
market.updated
signal.created
risk.decided
trade.intent
trade.executed
position.updated

Dashboard tabs

Tab	Fokus
Dashboard	equity, PnL, exposure, system health
Scanner	token/pool candidates
Token Intelligence	contract, holders, liquidity, whale flow
Signals	strategy scores dan alasan
Trade Terminal	quote, simulation, execute
Positions	PnL, TP/SL/trailing
Risk	risk score, circuit breaker
Network	RPC/WSS latency, block/Flashblock

Tab	Fokus
Logs	event dan execution audit

13. Observability, Fail-Safe, dan Performance Budget

Metric penting

- RPC/WSS latency dan error rate
- event-to-state latency
- strategy evaluation latency
- simulation latency
- signing latency
- submit-to-ack latency
- stale quote rejection
- failed/reverted transaction
- PnL, drawdown, win rate, expectancy

Performance budget contoh

Stage	Target internal awal
Event decode	< 1-2 ms
Market state update	< 1 ms
Strategy + risk	< 1-5 ms
Signing	< 1-5 ms
RPC / simulation	provider-dependent; biasanya dominan
End-to-end	ukur per provider dan strategi

Angka internal harus dianggap target engineering, bukan klaim performa jaringan. Ukur p50/p95/p99 secara nyata.

Emergency Stop

When triggered:

- New BUY: OFF
- Strategy auto-execution: OFF
- Monitoring: ON
- Controlled SELL / unwind: optionally ON
- Signer policy: restrictive mode

14. Roadmap MVP ke Production

Phase	Deliverable
1 - Observe	Rust RPC/WSS listener, decoder, market state, network monitor
2 - Intelligence	token/pool intelligence, risk scoring, scanner
3 - Paper Trading	strategy engine, replay/backtest, simulated positions
4 - Manual Live	quote, buy/sell simulation, manual execute
5 - Guarded Auto	small limits, circuit breaker, auto execution
6 - Low Latency	Flashblocks, optimized RPC, parallel simulation, local signer
7 - Advanced	multi-DEX routing, arbitrage, smart-money models, portfolio optimization

15. Checklist Implementasi

- Rust workspace dan CI tests aktif.
- Provider RPC/WSS diuji latency p50/p95/p99.
- Event gap/reorg handling terimplementasi.
- MarketState memiliki freshness timestamp dan stale-state rejection.
- Semua strategy mempunyai paper-trading mode.
- Risk engine dapat memblokir trade secara deterministik.
- Buy dan sell simulation diuji untuk token berisiko.
- Signer terisolasi dari dashboard/database.
- Position sizing dan max daily loss diterapkan.
- Emergency stop diuji.
- Metrics + structured logs tersedia sebelum live trading.
- Auto-trading dimulai dengan nominal sangat kecil dan limit ketat.

Target final: BASE -> Rust Real-Time Data -> In-Memory Intelligence -> Strategy -> Risk -> Simulation -> Signer -> Execution -> Position/Portfolio -> Analytics, dengan Next.js sebagai control plane.

Dokumen ini tidak menjanjikan keuntungan. Kecepatan eksekusi hanya satu faktor; kualitas sinyal, biaya, slippage, likuiditas, MEV, dan risk management tetap menentukan hasil.

16. Base Sepolia Testnet Integration

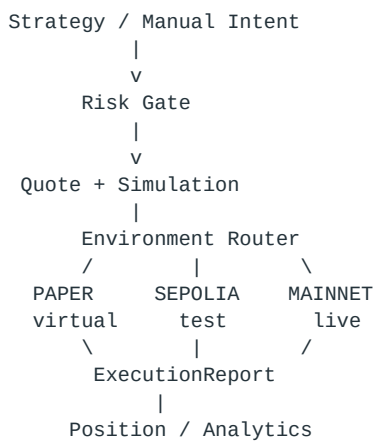
Dokumen v2.0 sudah memiliki **paper trading**, simulation, risk gate, dan live execution, tetapi belum memiliki tahap blockchain testnet eksplisit. Tambahan ini memasukkan **Base Sepolia** sebagai lingkungan wajib di antara paper trading dan Base Mainnet sehingga seluruh jalur transaksi dapat diuji end-to-end tanpa aset bernilai riil.

Prinsip v2.1: PAPER untuk menguji strategi, BASE SEPOLIA untuk menguji perilaku blockchain dan execution path, BASE MAINNET hanya setelah seluruh promotion gate lulus.

Matriks jaringan

Mode	Network	Chain ID	RPC HTTP	WSS / pre-confirmed	Nilai aset
PAPER	Tidak mengirim transaksi	-	Data provider / replay	-	Virtual
TESTNET	Base Sepolia	84532	https://sepolia.base.org	wss://sepolia.base.org atau wss://sepolia-preconf.base.org	Test ETH / test token
LIVE	Base Mainnet	8453	https://mainnet.base.org	wss://mainnet.base.org atau wss://mainnet-preconf.base.org	Real assets

Posisi Sepolia pada arsitektur



- Core logic strategy, risk, simulation result, signer policy, nonce manager, dan execution report tetap memakai domain type yang sama.
- Perbedaan lingkungan harus berasal dari konfigurasi network, contract registry, signer, limits, dan persistence namespace - bukan dari fork kode strategi.
- Setiap trade, order, signal, position, log, dan metric wajib memiliki field environment/network agar data testnet tidak tercampur dengan mainnet.

17. Environment, RPC, dan Rust Configuration

Konfigurasi environment harus eksplisit dan fail-closed. Aplikasi tidak boleh menebak network dari RPC URL saja. Saat boot, service memverifikasi **eth_chainId** / **net_version**, lalu menolak startup bila nilai aktual tidak sama dengan konfigurasi.

Contoh environment variables

```
APP_ENV=testnet
CHAIN_NAME=base-sepolia
CHAIN_ID=84532
RPC_HTTP=https://sepolia.base.org
RPC_WSS=wss://sepolia.base.org
FLASHBLOCKS_HTTP=https://sepolia-preconf.base.org
FLASHBLOCKS_WSS=wss://sepolia-preconf.base.org
SIGNER_PROFILE=sepolia-bot
MAX_TRADE_VALUE_USD=25
AUTO_EXECUTION=false
```

Rust domain configuration

```
enum TradingEnvironment {
    Paper,
    BaseSepolia,
    BaseMainnet,
}

struct ChainConfig {
    env: TradingEnvironment,
    chain_id: u64,
    rpc_http: Url,
    rpc_wss: Option<Url>,
    flashblocks_http: Option<Url>,
    flashblocks_wss: Option<Url>,
    confirmations_required: u64,
    max_trade_value: U256,
    auto_execution: bool,
}
```

Startup validation

Check	Sepolia expected	Fail action
eth_chainId / net_version	84532	Stop service; jangan sign/submit
Signer balance	Test ETH cukup untuk gas	Disable execution + alert
RPC health	latest block bergerak, latency sehat	Degraded / failover provider
Contract registry	Semua address bertag sepolia	Reject unresolved route
Block freshness	timestamp/height tidak stale	Risk gate = REJECT
Environment limits	testnet profile aktif	Reject config mismatch

Jangan reuse contract address Mainnet secara otomatis pada Sepolia. Router, factory, token, oracle, dan pool harus berasal dari registry per-network serta diverifikasi code-nya pada chain yang aktif.

18. Demo / Testnet Trading Flow

Base Sepolia bukan paper account. Transaksi tetap nyata pada blockchain testnet: wallet memiliki nonce, gas dibayar dengan test ETH, transaksi mempunyai hash, receipt, logs, status, dan dapat revert. Perbedaannya adalah aset testnet tidak dipakai sebagai aset bernilai riil.

Onboarding wallet testnet

- Buat wallet khusus testnet. Jangan gunakan seed/private key wallet produksi.
- Tambahkan Base Sepolia dengan Chain ID 84532 dan RPC resmi atau provider yang dipilih.
- Dapatkan test ETH melalui faucet yang tersedia dari ekosistem Base/provider. Faucet dapat berubah; tautan faucet harus diperlakukan sebagai konfigurasi/dokumentasi, bukan hard-code di trading core.
- Siapkan token uji. Untuk integration test paling deterministik, deploy ERC-20 test sendiri dan mint supply khusus pengujian.
- Jika menguji swap, deploy/konfigurasi pool/router testnet yang memang tersedia. Jangan mengasumsikan likuiditas mainnet tersedia di Sepolia.

Execution lifecycle di Sepolia

```
TradeIntent
|
RiskDecision::Pass
|
Quote / Route (SEPOLIA registry)
|
Buy Simulation + Sell Simulation
|
Allowance / Balance / Nonce check
|
Build tx with chain_id = 84532
|
Sepolia Signer
|
Submit -> tx hash
|
Known / Pre-confirmed / Included
|
Receipt + Logs + Position update
|
ExecutionReport(environment=BaseSepolia)
```

Test assets dan fixture strategy

Fixture	Tujuan
TEST-WETH / TEST-USDC	Happy-path swap dan PnL accounting
Token dengan transfer fee	Menguji quote vs amount-received dan sellability
Token yang dapat pause/revert	Menguji contract-risk gate dan simulation failure
Thin-liquidity pool	Menguji price impact, slippage, max position size
Large-wallet fixture	Menguji whale/smart-money event tanpa dana riil

19. Flashblocks dan Simulation pada Base Sepolia

Base menyediakan endpoint Standard dan Flashblocks untuk Sepolia. Pada tier Flashblocks, state **pending** merepresentasikan pre-confirmed block yang sedang dibangun dan diperbarui kira-kira setiap 200 ms. Ini cocok untuk menguji critical path low-latency blueprint sebelum mengaktifkannya di Mainnet.

Endpoint pengujian

Capability	Base Sepolia
Standard HTTP	https://sepolia.base.org
Standard WSS	wss://sepolia.base.org
Flashblocks HTTP	https://sepolia-preconf.base.org
Flashblocks WSS	wss://sepolia-preconf.base.org
Simulation	eth_simulateV1 pada Flashblocks endpoint
Tx mempool status	base_transactionStatus pada Flashblocks endpoint
Streams	newFlashblocks, newFlashblockTransactions, pendingLogs

Validation matrix sebelum submit

Stage	Metode	PASS condition
Read state	eth_getBalance / eth_call pending	Fresh + expected chain
Gas	eth_estimateGas + fee policy	Di bawah gas ceiling
Bundle simulation	eth_simulateV1	status success; no unexpected revert
Transfer trace	traceTransfers=true bila diperlukan	Net token/ETH flow sesuai expectation
Submit acknowledgement	base_transactionStatus	Known atau receipt terdeteksi
Pre-confirmation	newFlashblockTransactions / pendingLogs	Tx/log teramati
Final inclusion	eth_getTransactionReceipt	status=1 dan block included

Simulation bukan jaminan transaksi pasti sukses. State dapat berubah setelah simulation; bot tetap harus menerapkan quote TTL, nonce discipline, slippage limit, replacement policy, dan post-submit monitoring.

Load/latency test yang wajib dilakukan di Sepolia

- Ukur p50/p95/p99 RPC HTTP dan WSS untuk provider yang akan dipakai.
- Ukur event -> state -> strategy -> risk -> simulation -> sign -> submit secara end-to-end.
- Inject disconnect, timeout, duplicate event, sequence gap, stale block, dan RPC mismatch.
- Uji burst Flashblocks agar bounded channel/backpressure tidak membuat core service kehilangan kendali.

20. Database, Dashboard, dan Observability untuk Testnet

Tambahkan dimensi environment sejak schema awal. Ini mencegah statistik, posisi, dan audit trail testnet terlihat sebagai data produksi.

Schema minimum

```
network_env      ENUM('paper', 'base_sepolia', 'base_mainnet')
chain_id         BIGINT
wallet_profile   TEXT
contract_registry TEXT / version
provider_id      TEXT
execution_mode   ENUM('simulate', 'manual', 'auto')
```

```
-- apply to:
signals, risk_events, orders, trades, positions,
transactions, execution_reports, system_logs
```

Dashboard additions

Area	Penambahan v2.1
Global header	Environment selector + badge PAPER / TESTNET / LIVE
Trade Terminal	Chain ID, signer profile, RPC provider, simulation result
Network	Separate latency/error charts per environment/provider
Positions	Filter environment; testnet PnL tidak masuk production equity
Logs	network_env + chain_id + tx hash + provider + correlation ID
Risk	Limits profile yang aktif dan alasan environment rejection
Settings	Contract registry version + explicit mainnet unlock control

Visual safety rules

- PAPER: label permanen dan tombol action menyebut “Simulate”, bukan “Buy/Sell” tanpa konteks.
- TESTNET: badge kuning/oranye permanen, tampilkan “Base Sepolia - Chain 84532” dekat tombol submit.
- LIVE: badge merah/critical, nominal fiat/asset nyata, dan confirmation step tambahan untuk manual mode.
- Jangan mengandalkan warna saja; selalu tampilkan teks environment dan chain ID.

Metrics tambahan

```
rpc_latency_ms{env,provider,method}
rpc_error_total{env,provider}
flashblock_gap_total{env}
simulation_revert_total{env,reason}
submit_to_known_ms{env}
known_to_receipt_ms{env}
chain_mismatch_total{service}
testnet_trade_total{strategy,result}
```

21. Promotion Gates: Paper -> Sepolia -> Mainnet

Roadmap v2.0 menempatkan Paper Trading lalu Manual Live. v2.1 menambahkan Base Sepolia sebagai fase wajib supaya wallet, signer, approvals, nonce, route, simulation, submit, receipt, dan failure recovery diuji pada jaringan EVM nyata sebelum aset riil digunakan.

Roadmap revisi

Phase	Deliverable / gate
1 - Observe	Rust RPC/WSS listener, decoder, market state, network monitor
2 - Intelligence	token/pool intelligence, risk scoring, scanner
3 - Paper Trading	strategy engine, replay/backtest, virtual positions; no signing
4 - Base Sepolia Manual	test wallet, contract registry, quote, buy/sell simulation, manual submit
5 - Base Sepolia Guarded Auto	small test limits, auto execution, circuit breaker, failure injection
6 - Mainnet Manual	production signer isolated, explicit unlock, very small real-size trades
7 - Mainnet Guarded Auto	strict limits, circuit breaker, full observability, staged strategy enablement
8 - Low Latency	Flashblocks tuning, optimized provider, parallel simulation, local signer policy
9 - Advanced	multi-DEX routing, arbitrage, smart-money models, portfolio optimization

Sepolia exit criteria

- 100% network/chain-id validation coverage; intentional mismatch test harus fail-closed.
- Buy dan sell happy-path berhasil, termasuk allowance, nonce, receipt, event decoding, dan position update.
- Revert, insufficient balance, stale quote, slippage breach, RPC timeout, duplicate event, dan dropped/replaced transaction sudah diuji.
- Emergency stop mampu memblokir new BUY dan auto execution tanpa mematikan monitoring.
- Environment isolation terverifikasi: tidak ada key, DB row, contract address, metrics namespace, atau UI state Sepolia yang dapat dipakai sebagai Mainnet secara implisit.
- Performance p50/p95/p99 dan error budget terdokumentasi per provider; provider failover diuji bila digunakan.
- Mainnet tetap terkunci sampai operator melakukan explicit promotion/unlock.

Promotion ke Mainnet adalah perubahan environment berisiko tinggi. Jangan membuat “toggle sekali klik” yang diam-diam mengganti testnet menjadi live; gunakan approval yang eksplisit, policy signer berbeda, dan secrets berbeda.

22. Checklist Implementasi v2.1 dan Referensi

Checklist tambahan Base Sepolia

- [] Base Sepolia (84532) tersedia sebagai TradingEnvironment terpisah.
- [] RPC Standard dan Flashblocks Sepolia dikonfigurasi melalui secret/config manager.
- [] Boot-time chain verification memblokir chain mismatch.
- [] Signer profile Sepolia berbeda dari signer Mainnet.
- [] Contract registry dipisahkan per chain dan tidak memiliki fallback lintas network.
- [] Database dan metrics memiliki network_env / chain_id.
- [] UI menampilkan environment + chain ID di semua action yang dapat mengirim transaksi.
- [] Test ETH/test token onboarding terdokumentasi.
- [] Buy + sell simulation diuji menggunakan eth_simulateV1 bila Flashblocks digunakan.
- [] newFlashblocks/newFlashblockTransactions/pendingLogs diuji bila menjadi bagian critical path.
- [] base_transactionStatus / receipt monitoring memiliki timeout dan fallback policy.
- [] Paper -> Sepolia -> Mainnet promotion gates diimplementasikan di CI/release runbook.
- [] Sepolia failure-injection suite lulus sebelum Mainnet unlock.

Referensi resmi yang digunakan untuk addendum ini

- Base RPC Overview - network IDs dan endpoint Standard/Flashblocks:
<https://docs.base.org/base-chain/api-reference/rpc-overview>
- Base Flashblocks API Overview - pre-confirmed state dan subscription types:
<https://docs.base.org/base-chain/api-reference/flashblocks-api/flashblocks-api-overview>
- eth_simulateV1 - simulation terhadap pre-confirmed state:
https://docs.base.org/base-chain/api-reference/flashblocks-api/eth_simulateV1
- base_transactionStatus - status penerimaan transaksi pada Flashblocks endpoint:
https://docs.base.org/base-chain/api-reference/flashblocks-api/base_transactionStatus
- newFlashblocks / newFlashblockTransactions / pendingLogs - WebSocket streams untuk pre-confirmation.

Target final v2.1: BASE DATA -> Rust Real-Time State -> Strategy -> Risk -> Simulation -> Environment Router -> Paper / Base Sepolia / Base Mainnet -> Signer -> Execution -> Position/Portfolio -> Analytics, dengan Next.js sebagai control plane.

Catatan: Base Sepolia dipakai untuk validation dan engineering test, bukan untuk membuktikan profitability. Profitability tetap harus dievaluasi melalui data historis, paper trading, biaya aktual, slippage, likuiditas, MEV, dan risk management.